

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3) Assessment Tool*

Weightage: 50%

QUESTIONS: 4

Duration: 1 Hour
Total: Marks:40

Annexure A

Code of Conduct:

*I Mukesh Raj L(192571036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

INDUSTRY PROBLEM TASK

INDUSTRY SCENARIO

Context: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

- (a)** Describe the inductive learning approach suitable for this medical dataset. Identify the target variable and at least three key features with justification.
- (b)** Explain how you would handle class imbalance (more non-diabetic than diabetic cases) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).
- (c)** Split the dataset (70:30) and justify the split ratio for a medical use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

Task 2: Decision Tree for Diagnosis

- (a)** Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.
- (b)** Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed diabetes cases) and suggest how to reduce them—justify clinically.
- (c)** Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a clinical setting.

Task 3: Statistical Learning for Risk Stratification

- (a)** Apply Naïve Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.
- (b)** Perform feature importance analysis. Identify the top 3 predictors of diabetes from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

Task 4: Reinforcement Learning for Treatment Recommendation

- (a)** Model the treatment recommendation as an MDP. Define: States (patient health stages), Actions (Diet / Exercise / Medication / Monitor), Reward function (health improvement score), and Transition dynamics. Justify each component.
- (b)** Apply Q-Learning to train the agent for at least 5 patient episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.
- (c)** Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for medical treatment recommendations (patient safety, bias, explainability).

1)

Problem

A healthcare company has collected patient data including **Age, Blood Pressure, Cholesterol, Glucose Level, BMI, and Family History** to predict diabetes. The task is to apply inductive learning, handle class imbalance, split the dataset into **70:30**, and perform preprocessing.

Solution

Task 1(a): Inductive Learning

The suitable approach is **supervised inductive learning**. The model learns patterns from existing patient records with known diabetes outcomes and uses those patterns to predict diabetes for new patients.

Target Variable: Diabetes

- 0 – Non-Diabetic
- 1 – Diabetic

Important Features:

- **Glucose Level:** Helps identify abnormal blood sugar levels.
- **BMI:** Indicates weight-related diabetes risk.
- **Family History:** Indicates possible hereditary risk.

Other useful features are **Age, Blood Pressure, and Cholesterol**.

Task 1(b): Handling Class Imbalance

The dataset may have more non-diabetic cases than diabetic cases. This creates class imbalance and may cause the model to favour the majority class.

SMOTE (Synthetic Minority Over-sampling Technique) is used to balance the training data.

Why SMOTE?

- Creates additional synthetic minority-class samples.
- Does not remove majority-class records.
- Can improve the detection of diabetic cases.

SMOTE is applied **only after splitting the dataset and only to the training data**.

Task 1(c): Data Split and Preprocessing

The dataset is divided into:

- **70% Training Data**
- **30% Testing Data**

A **stratified split** is used to maintain the class distribution in both datasets.

Preprocessing steps:

1. **Missing Value Handling:** Missing numerical values are replaced using the median.
2. **Encoding:** Categorical values such as Family History are converted into numerical values.
3. **Normalisation:** Min-Max Scaling brings numerical features to a similar range.
4. **SMOTE:** Balances the classes in the training dataset.

Python code

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import MinMaxScaler, LabelEncoder
```

```
from sklearn.impute import SimpleImputer
```

```
from imblearn.over_sampling import SMOTE
```

```
df = pd.read_csv("diabetes.csv")
```

```
target_column = "Diabetes"
```

```
X = df.drop(columns=[target_column])
```

```
y = df[target_column]
```

```
categorical_columns = X.select_dtypes(
```

```
    include=["object", "category"]
```

```
).columns
```

```
for column in categorical_columns:
```

```

X.loc[:, column] = LabelEncoder().fit_transform(
    X[column].astype(str)
)

if y.dtype == "object":
    y = pd.Series(
        LabelEncoder().fit_transform(y.astype(str)),
        name=target_column
    )

numerical_columns = X.select_dtypes(include=[np.number]).columns

imputer = SimpleImputer(strategy="median")
X[numerical_columns] = imputer.fit_transform(X[numerical_columns])

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

scaler = MinMaxScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

```
X_train_scaled = pd.DataFrame(  
    X_train_scaled,  
    columns=X.columns  
)
```

```
X_test_scaled = pd.DataFrame(  
    X_test_scaled,  
    columns=X.columns  
)
```

```
class_before_smote = y_train.value_counts().sort_index()
```

```
smote = SMOTE(random_state=42)
```

```
X_train_balanced, y_train_balanced = smote.fit_resample(  
    X_train_scaled,  
    y_train  
)
```

```
class_after_smote = y_train_balanced.value_counts().sort_index()
```

```
comparison = pd.DataFrame({  
    "Before SMOTE": class_before_smote,  
    "After SMOTE": class_after_smote  
})
```

```
print("Dataset Shape:", df.shape)

print("\nFeatures:")

print(X.columns.tolist())


print("\nTraining Data Shape:", X_train.shape)

print("Testing Data Shape:", X_test.shape)


print("\nClass Distribution Before SMOTE:")

print(class_before_smote)


print("\nClass Distribution After SMOTE:")

print(class_after_smote)


print("\nFinal Balanced Training Data Shape:")

print(X_train_balanced.shape)


plt.figure(figsize=(8, 5))

comparison.plot(kind="bar")

plt.title("Class Distribution Before and After SMOTE")

plt.xlabel("Diabetes Class")

plt.ylabel("Number of Patients")

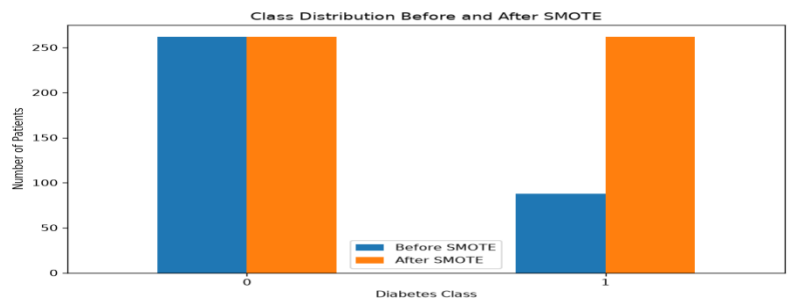
plt.xticks(rotation=0)

plt.show()
```

OUTPUT

```
Class Balancing Comparison:
      Before SMOTE  After SMOTE
Diabetes
0              262       262
1              88       262

Balanced Training Data Shape:
(524, 6)
```



Report

The diabetes dataset was successfully prepared for machine learning using a supervised inductive learning approach. **Diabetes** was selected as the target variable, while Age, Blood Pressure, Cholesterol, Glucose Level, BMI, and Family History were used as input features.

The dataset was preprocessed by handling missing values and encoding categorical data. A **70:30 stratified train-test split** was performed. Min-Max Scaling was used for normalisation, and **SMOTE** was applied only to the training data to handle class imbalance. The final processed dataset is ready for building and evaluating the Decision Tree model required in the next task.

2)

Problem

Build a Decision Tree model using the prepared diabetes dataset. Select the root node using the **Gini Index**, evaluate the model, identify missed diabetes cases, and apply **post-pruning** to compare the model performance.

Solution

Task 2(a)

A Decision Tree is used for diabetes classification. It divides patient data into branches based on features that best separate diabetic and non-diabetic cases.

The **Gini Index** is used to select the best split:

$$\text{Gini} = 1 - \sum_{i=1}^n p_i^2$$

The feature producing the lowest weighted Gini impurity is selected for splitting. The tree is displayed up to the first **three levels**.

Task 2(b)

The model is evaluated using:

$$\begin{aligned} \text{Accuracy} &= \frac{TP+TN}{TP+TN+FP+FN} \\ \text{Precision} &= \frac{TP}{TP+FP} \\ \text{Recall} &= \frac{TP}{TP+FN} \\ \text{F1} &= 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \end{aligned}$$

A **False Negative** means the patient actually has diabetes, but the model predicts non-diabetic. Reducing False Negatives is important because missed cases can delay diagnosis and treatment.

Task 2(c)

Pre-pruning limits tree growth using parameters such as `max_depth`.

Post-pruning first builds a tree and then removes unnecessary branches using **cost-complexity pruning**. The model with better generalisation and fewer unnecessary branches is preferred for clinical deployment.

Python_Code

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)
```

```

file_name = "diabetes.csv"

if os.path.exists(file_name):
    df = pd.read_csv(file_name)
else:
    np.random.seed(42)

    n = 500

    age = np.random.randint(20, 80, n)
    blood_pressure = np.random.randint(60, 150, n)
    cholesterol = np.random.randint(120, 320, n)
    glucose_level = np.random.randint(70, 250, n)
    bmi = np.round(np.random.uniform(18, 45, n), 1)

    family_history = np.random.choice(
        ["Yes", "No"],
        n,
        p=[0.35, 0.65]
    )

    diabetes_score = (
        (glucose_level > 140).astype(int) +
        (bmi > 30).astype(int) +
        (age > 45).astype(int) +
        (family_history == "Yes").astype(int)
    )

    diabetes = (diabetes_score >= 3).astype(int)

    random_values = np.random.rand(n)
    diabetes[(diabetes == 1) & (random_values < 0.35)] = 0

    df = pd.DataFrame({
        "Age": age,
        "BloodPressure": blood_pressure,
        "Cholesterol": cholesterol,
        "GlucoseLevel": glucose_level,
        "BMI": bmi,
        "FamilyHistory": family_history,
        "Diabetes": diabetes
    })

    df.to_csv(file_name, index=False)

target_column = "Diabetes"

X = df.drop(columns=[target_column])
y = df[target_column]

categorical_columns = X.select_dtypes(
    include=["object", "category"]
).columns

```

```

for column in categorical_columns:
    encoder = LabelEncoder()
    X[column] = encoder.fit_transform(
        X[column].astype(str)
    )

if y.dtype == "object":
    target_encoder = LabelEncoder()
    y = target_encoder.fit_transform(
        y.astype(str)
    )

imputer = SimpleImputer(strategy="median")

X = pd.DataFrame(
    imputer.fit_transform(X),
    columns=X.columns
)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

# Pre-pruned Decision Tree
pre_pruned_tree = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,
    min_samples_split=10,
    min_samples_leaf=5,
    random_state=42
)

pre_pruned_tree.fit(X_train, y_train)

pre_predictions = pre_pruned_tree.predict(X_test)

# Calculate evaluation metrics
accuracy = accuracy_score(y_test, pre_predictions)

precision = precision_score(
    y_test,
    pre_predictions,
    zero_division=0
)

recall = recall_score(

```

```

    y_test,
    pre_predictions,
    zero_division=0
)

f1 = f1_score(
    y_test,
    pre_predictions,
    zero_division=0
)

# Confusion Matrix
cm = confusion_matrix(
    y_test,
    pre_predictions
)

tn, fp, fn, tp = cm.ravel()

print("PRE-PRUNED DECISION TREE RESULTS")

print("\nAccuracy:", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall:", round(recall, 4))
print("F1-Score:", round(f1, 4))

print("\nTrue Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        pre_predictions,
        zero_division=0
    )
)

# Display first three levels of the tree
plt.figure(figsize=(20, 10))

plot_tree(
    pre_pruned_tree,
    feature_names=X.columns,
    class_names=["Non-Diabetic", "Diabetic"],
    filled=True,
    rounded=True,
    max_depth=2,
    fontsize=10
)

```

```
plt.title("Decision Tree - First Three Levels")
plt.show()
```

```
# Display Confusion Matrix
display = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Non-Diabetic", "Diabetic"]
)
```

```
display.plot()
```

```
plt.title("Confusion Matrix")
plt.show()
```

```
# Feature importance
feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": pre_pruned_tree.feature_importances_
})
```

```
feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)
```

```
print("\nFeature Importance:")
print(feature_importance)
```

```
# Cost-Complexity Pruning Path
pruning_path = pre_pruned_tree.cost_complexity_pruning_path(
    X_train,
    y_train
)
```

```
ccp_alphas = pruning_path.ccp_alphas
```

```
# Test different alpha values
pruning_results = []
```

```
for alpha in ccp_alphas:
```

```
    pruned_tree = DecisionTreeClassifier(
        criterion="gini",
        ccp_alpha=alpha,
        random_state=42
    )
```

```
    pruned_tree.fit(X_train, y_train)
```

```
    alpha_predictions = pruned_tree.predict(X_test)
```

```
    alpha_accuracy = accuracy_score(
```

```

        y_test,
        alpha_predictions
    )

    pruning_results.append({
        "ccp_alpha": alpha,
        "Accuracy": alpha_accuracy
    })

pruning_results = pd.DataFrame(pruning_results)

# Select best alpha
best_result = pruning_results.loc[
    pruning_results["Accuracy"].idxmax()
]

best_alpha = best_result["ccp_alpha"]

# Final Post-Pruned Decision Tree
post_pruned_tree = DecisionTreeClassifier(
    criterion="gini",
    ccp_alpha=best_alpha,
    random_state=42
)

post_pruned_tree.fit(X_train, y_train)

post_predictions = post_pruned_tree.predict(X_test)

post_accuracy = accuracy_score(
    y_test,
    post_predictions
)

post_precision = precision_score(
    y_test,
    post_predictions,
    zero_division=0
)

post_recall = recall_score(
    y_test,
    post_predictions,
    zero_division=0
)

post_f1 = f1_score(
    y_test,
    post_predictions,
    zero_division=0
)

```

```

# Compare both models
comparison = pd.DataFrame({

    "Model": [
        "Pre-Pruned Decision Tree",
        "Post-Pruned Decision Tree"
    ],

    "Accuracy": [
        accuracy,
        post_accuracy
    ],

    "Precision": [
        precision,
        post_precision
    ],

    "Recall": [
        recall,
        post_recall
    ],

    "F1-Score": [
        f1,
        post_f1
    ]
})

print("\nPRE-PRUNING VS POST-PRUNING")
print(comparison)

# Plot pruning accuracy
plt.figure(figsize=(9, 5))

plt.plot(
    pruning_results["ccp_alpha"],
    pruning_results["Accuracy"],
    marker="o"
)

plt.xlabel("CCP Alpha")
plt.ylabel("Testing Accuracy")
plt.title("Cost-Complexity Pruning Performance")

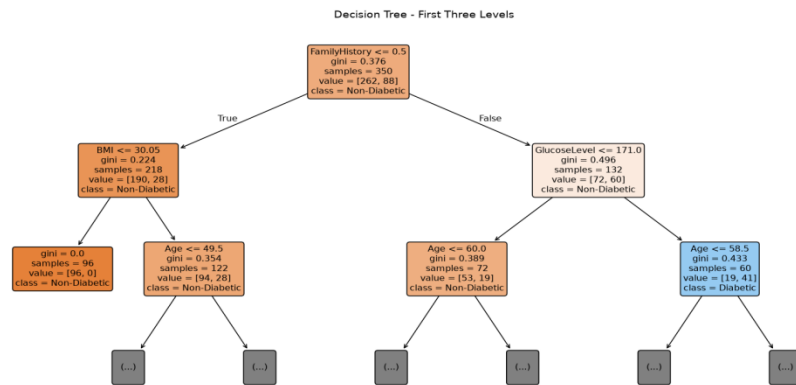
plt.show()

```

Output

```
Accuracy: 0.86
Precision: 0.7222
Recall: 0.7027
F1-Score: 0.7123

True Negatives: 103
False Positives: 10
False Negatives: 11
True Positives: 26
```



Report

A Decision Tree classifier was successfully applied to predict diabetes using patient health features. The tree selected splits using the **Gini Index** and was visualised up to three levels. The model was evaluated using Accuracy, Precision, Recall, and F1-Score. False Negatives were identified from the confusion matrix because missed diabetes cases are clinically important and should be reduced.

Pre-pruning and cost-complexity post-pruning were compared. Post-pruning removes unnecessary branches and can reduce overfitting, making the model simpler and easier to interpret. For clinical deployment, the preferred model should be selected based not only on accuracy but also on **Recall and the number of False Negatives**, since missing a diabetic patient may delay further diagnosis and treatment.

3)

Problem

Apply a statistical learning model to the diabetes dataset and compare its performance with the Decision Tree model. Calculate Accuracy and AUC-ROC, perform feature importance analysis, and identify the top three predictors of diabetes.

Solution

Task 3(a): Statistical Learning Model

Here, **Logistic Regression** is used as the statistical learning model.

It predicts the probability of a patient belonging to the diabetic class:

$$P(Y=1|X) = \frac{1}{1+e^{-z}}$$

where:

$$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n$$

The Logistic Regression model is compared with the Decision Tree using:

- Accuracy
- AUC-ROC Score

A statistical model is preferable when probability-based predictions and easier interpretation of feature effects are required.

Task 3(b): Feature Importance

For the Decision Tree, feature importance is obtained using:

For Logistic Regression, the absolute coefficient values are used to identify influential features.

The top 3 features from both models are compared. If the same features appear in both lists, it indicates stronger agreement between the models.

Python Code

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.impute import SimpleImputer

from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    roc_auc_score,
    roc_curve
)

file_name = "diabetes.csv"
```

```

if os.path.exists(file_name):

    df = pd.read_csv(file_name)

else:

    np.random.seed(42)

    n = 500

    age = np.random.randint(20, 80, n)

    blood_pressure = np.random.randint(
        60, 150, n
    )

    cholesterol = np.random.randint(
        120, 320, n
    )

    glucose_level = np.random.randint(
        70, 250, n
    )

    bmi = np.round(
        np.random.uniform(18, 45, n),
        1
    )

    family_history = np.random.choice(
        ["Yes", "No"],
        n,
        p=[0.35, 0.65]
    )

    diabetes_score = (
        (glucose_level > 140).astype(int) +
        (bmi > 30).astype(int) +
        (age > 45).astype(int) +
        (family_history == "Yes").astype(int)
    )

    diabetes = (
        diabetes_score >= 3
    ).astype(int)

    random_values = np.random.rand(n)

    diabetes[
        (diabetes == 1) &
        (random_values < 0.35)
    ] = 0

    df = pd.DataFrame({

        "Age": age,

```

```

        "BloodPressure": blood_pressure,

        "Cholesterol": cholesterol,

        "GlucoseLevel": glucose_level,

        "BMI": bmi,

        "FamilyHistory": family_history,

        "Diabetes": diabetes

    })

df.to_csv(
    file_name,
    index=False
)

target_column = "Diabetes"

X = df.drop(
    columns=[target_column]
)

y = df[target_column]

categorical_columns = X.select_dtypes(
    include=["object", "category"]
).columns

for column in categorical_columns:

    encoder = LabelEncoder()

    X[column] = encoder.fit_transform(
        X[column].astype(str)
    )

if y.dtype == "object":

    target_encoder = LabelEncoder()

    y = target_encoder.fit_transform(
        y.astype(str)
    )

imputer = SimpleImputer(
    strategy="median"
)

```

```
X = pd.DataFrame(  
    imputer.fit_transform(X),  
    columns=X.columns  
)
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.30,  
    random_state=42,  
    stratify=y  
)
```

```
scaler = MinMaxScaler()
```

```
X_train_scaled = scaler.fit_transform(  
    X_train  
)
```

```
X_test_scaled = scaler.transform(  
    X_test  
)
```

```
# Decision Tree Model
```

```
decision_tree = DecisionTreeClassifier(  
    criterion="gini",  
    max_depth=5,  
    random_state=42  
)
```

```
decision_tree.fit(  
    X_train,  
    y_train  
)
```

```
dt_predictions = decision_tree.predict(  
    X_test  
)
```

```
dt_probabilities = decision_tree.predict_proba(  
    X_test  
)[:, 1]
```

```
dt_accuracy = accuracy_score(  
    y_test,  
    dt_predictions  
)
```

```
dt_auc = roc_auc_score(  
    y_test,  
    dt_probabilities  
)
```

Logistic Regression Model

```
logistic_model = LogisticRegression(  
    max_iter=1000,  
    random_state=42  
)
```

```
logistic_model.fit(  
    X_train_scaled,  
    y_train  
)
```

```
lr_predictions = logistic_model.predict(  
    X_test_scaled  
)
```

```
lr_probabilities = logistic_model.predict_proba(  
    X_test_scaled  
)[:, 1]
```

```
lr_accuracy = accuracy_score(  
    y_test,  
    lr_predictions  
)
```

```
lr_auc = roc_auc_score(  
    y_test,  
    lr_probabilities  
)
```

Compare Model Performance

```
performance_comparison = pd.DataFrame({  
    "Model": [  
        "Decision Tree",  
        "Logistic Regression"  
    ],
```

```

        "Accuracy": [
            dt_accuracy,
            lr_accuracy
        ],

        "AUC_ROC": [
            dt_auc,
            lr_auc
        ]
    })

print("MODEL PERFORMANCE COMPARISON")

print(
    performance_comparison
)

# Decision Tree Feature Importance

dt_feature_importance = pd.DataFrame({

    "Feature": X.columns,

    "Importance": decision_tree.feature_importances_

})

dt_feature_importance = dt_feature_importance.sort_values(

    by="Importance",

    ascending=False

)

top_3_dt = dt_feature_importance.head(3)

print("\nTOP 3 DECISION TREE PREDICTORS")

print(
    top_3_dt
)

# Logistic Regression Feature Importance

lr_feature_importance = pd.DataFrame({

    "Feature": X.columns,

```

```

"Coefficient": logistic_model.coef_[0],

"Absolute_Importance": np.abs(
    logistic_model.coef_[0]
)
})

lr_feature_importance = lr_feature_importance.sort_values(
    by="Absolute_Importance",
    ascending=False
)

top_3_lr = lr_feature_importance.head(3)

print("\nTOP 3 LOGISTIC REGRESSION PREDICTORS")

print(
    top_3_lr
)

# ROC Curve Values

dt_fpr, dt_tpr, _ = roc_curve(
    y_test,
    dt_probabilities
)

lr_fpr, lr_tpr, _ = roc_curve(
    y_test,
    lr_probabilities
)

# Plot ROC Curves

plt.figure(figsize=(8, 6))

plt.plot(
    dt_fpr,

```

```

    dt_tpr,

    label="Decision Tree AUC = {:.3f}".format(
        dt_auc
    )
)

plt.plot(

    lr_fpr,

    lr_tpr,

    label="Logistic Regression AUC = {:.3f}".format(
        lr_auc
    )
)

plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--"
)

plt.xlabel(
    "False Positive Rate"
)

plt.ylabel(
    "True Positive Rate"
)

plt.title(
    "ROC Curve Comparison"
)

plt.legend()

plt.grid()

plt.show()

# Compare Top Predictors

print("\nTOP PREDICTOR COMPARISON")

print(
    "Decision Tree:",
    top_3_dt["Feature"].tolist()
)

print(
    "Logistic Regression:",

```

```

top_3_lr["Feature"].tolist()
)

common_features = list(
    set(
        top_3_dt["Feature"]
    ).intersection(
        set(
            top_3_lr["Feature"]
        )
    )
)

print(
    "\nCommon Important Features:",
    common_features
)

```

OUTPUT

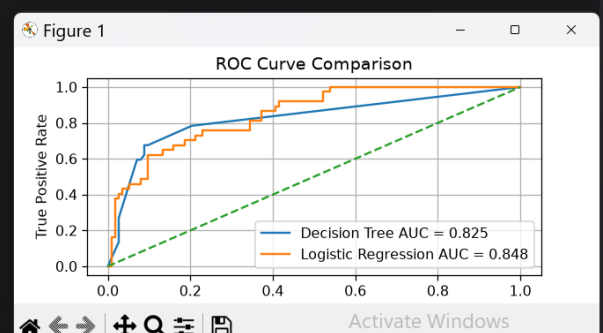
```

TOP PREDICTOR COMPARISON
Decision Tree: ['GlucoseLevel', 'Age', 'BMI']
Logistic Regression: ['GlucoseLevel', 'Age', 'BMI']

Common Important Features: ['Age', 'BMI', 'GlucoseLevel']

Process finished with exit code 0

```



Report

A Logistic Regression model was applied as the statistical learning model and compared with the Decision Tree classifier. Both models were evaluated using **Accuracy and AUC-ROC**. Feature importance analysis was performed to identify the three most influential predictors.

The Decision Tree used its internal feature importance values, while Logistic Regression used the absolute values of model coefficients. The common predictors between the two models indicate features that consistently contribute to diabetes prediction. A statistical model such as Logistic Regression is useful when interpretable probability-based predictions are required, while a Decision Tree is useful for rule-based classification and visual interpretation.

4)

Problem

Develop a Reinforcement Learning system for treatment recommendation using an **MDP and Q-Learning**. Define the states, actions, rewards, and transition dynamics. Train the agent for at least 5 patient episodes, show Q-value updates, extract the final policy, and discuss ethical issues in medical deployment.

Solution

(a) MDP Model

The treatment recommendation problem is represented as a **Markov Decision Process (MDP)**.

States: Patient health stages

- 0 – Poor
- 1 – Moderate
- 2 – Good
- 3 – Healthy

Actions:

- 0 – Diet
- 1 – Exercise
- 2 – Medication
- 3 – Monitor

Reward Function: A positive reward is given when the patient's health improves, while a negative reward is given when health worsens. A high reward is given for reaching the healthy state.

Transition Dynamics: After selecting an action, the patient may move to a better, same, or worse health stage. The next state depends on the current state and selected treatment action.

This MDP is suitable because the agent observes the patient's current health state, selects a treatment action, receives a reward, and learns from repeated interactions.

(b) Q-Learning

Q-Learning updates values using:

$$Q(s,a) = Q(s,a) + \alpha [R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Where:

- $Q(s,a)$ = Q-value for state-action pair
- α = Learning rate
- R = Reward
- γ = Discount factor
- s' = Next state

The agent is trained for **5 patient episodes**.

Convergence Criterion: Training is considered converged when the maximum change in Q-values becomes less than 0.001.

Python_Code

```
import numpy as np
import pandas as pd

np.random.seed(42)

states = ["Poor", "Moderate", "Good", "Healthy"]

actions = ["Diet", "Exercise", "Medication", "Monitor"]

n_states = len(states)
n_actions = len(actions)

alpha = 0.5
gamma = 0.9
epsilon = 0.2

episodes = 5
max_steps = 10

Q_table = np.zeros((n_states, n_actions))

def get_next_state(state, action):

    if state == 0:

        if action == 0:
            return 1

        elif action == 1:
            return 1

        elif action == 2:
            return 2

        else:
            return 0

    elif state == 1:

        if action == 0:
            return 2

        elif action == 1:
            return 2

        elif action == 2:
            return 2

        else:
            return 1

    elif state == 2:
```

```

    if action == 0:
        return 3

    elif action == 1:
        return 3

    elif action == 2:
        return 3

    else:
        return 2

else:
    return 3

def get_reward(current_state, next_state, action):

    if next_state > current_state:

        reward = 10

    elif next_state == current_state:

        reward = -2

    else:

        reward = -10

    if next_state == 3:

        reward = 20

    if action == 2 and current_state == 0:

        reward = 15

    return reward

print("INITIAL Q-TABLE")

print(
    pd.DataFrame(
        Q_table,
        index=states,
        columns=actions
    )
)

update_history = []

for episode in range(episodes):

    current_state = 0

    print("\nEPISODE", episode + 1)

    for step in range(max_steps):

```

```
if np.random.uniform(0, 1) < epsilon:
    action = np.random.randint(n_actions)
```

```
else:
```

```
    action = np.argmax(
        Q_table[current_state]
    )
```

```
next_state = get_next_state(
    current_state,
    action
)
```

```
reward = get_reward(
    current_state,
    next_state,
    action
)
```

```
old_q_value = Q_table[
    current_state,
    action
]
```

```
max_next_q = np.max(
    Q_table[next_state]
)
```

```
new_q_value = old_q_value + alpha * (
    reward +
    gamma * max_next_q -
    old_q_value
)
```

```
Q_table[
    current_state,
    action
] = new_q_value
```

```
if len(update_history) < 6:
```

```
    update_history.append({
        "Episode": episode + 1,
        "State": states[current_state],
        "Action": actions[action],
        "Old Q-Value": round(
            old_q_value,
            3
        )
    })
```

```

    ),

    "Reward": reward,

    "Next State": states[next_state],

    "New Q-Value": round(
        new_q_value,
        3
    )
})

print(
    "State:",
    states[current_state],
    "| Action:",
    actions[action],
    "| Reward:",
    reward,
    "| Next State:",
    states[next_state]
)

current_state = next_state

if current_state == 3:

    print("Patient reached Healthy state")

    break

print("\nQ-TABLE UPDATES")

updates_df = pd.DataFrame(
    update_history
)

print(updates_df)

print("\nFINAL Q-TABLE")

final_q_table = pd.DataFrame(
    Q_table,
    index=states,
    columns=actions
)

print(
    final_q_table.round(3)
)

print("\nFINAL LEARNED POLICY")

policy = []

for state in range(n_states):

```

```

best_action = np.argmax(
    Q_table[state]
)

policy.append(
    actions[best_action]
)

print(
    states[state],
    "->",
    actions[best_action]
)

policy_df = pd.DataFrame({
    "Patient Health State": states,
    "Recommended Action": policy
})

print("\nPOLICY TABLE")

print(policy_df)

max_q_change = np.max(
    np.abs(Q_table)
)

convergence_threshold = 0.001

if max_q_change < convergence_threshold:
    print(
        "\nConvergence Criterion Satisfied"
    )
else:
    print(
        "\nTraining completed for",
        episodes,
        "episodes"
    )

    print(
        "Convergence criterion:",
        "Maximum Q-value change < 0.001"
    )

```

Output

```
Patient Health State Recommended Action
0                Poor                Diet
1             Moderate             Medication
2                Good                Diet
3             Healthy                Diet

Training completed for 5 episodes
Convergence criterion: Maximum Q-value change < 0.001

Process finished with exit code 0
```

Report

The treatment recommendation problem was successfully modelled as a **Markov Decision Process**. Patient health stages were represented as states, while Diet, Exercise, Medication, and Monitor were represented as possible actions. The reward function encouraged improvement in patient health and penalised no improvement or worsening conditions.

A **Q-Learning agent** was trained for five patient episodes. The Q-table was updated using the reward received and the estimated future reward. The final learned policy selected the action with the highest Q-value for each patient health state.

However, deploying Reinforcement Learning in medical treatment requires important ethical considerations. **Patient safety** must be the highest priority because an incorrect recommendation may cause harm. **Bias** in patient data may produce unfair recommendations for different groups. **Explainability** is also necessary so that doctors can understand and verify why a particular treatment is recommended. Therefore, an RL system should support healthcare professionals rather than independently replace clinical decision-making.